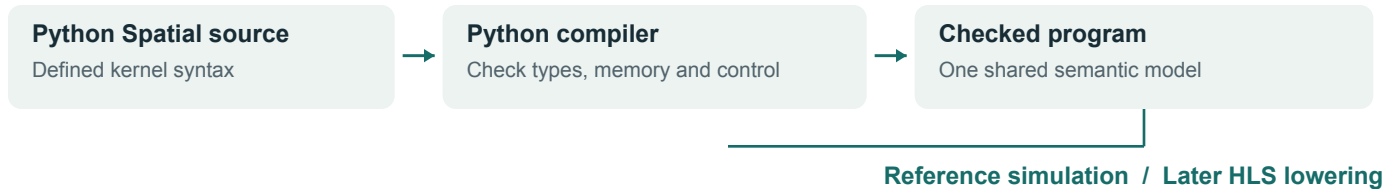


A Python compiler for Spatial

Design, implementation progress and next steps

A Python rewrite can put Spatial's frontend, compiler and reference simulator in Python. The proposed design preserves explicit types, memories and control, with one checked program representation shared by simulation and later HLS lowering. We have completed the initial architecture research and begun implementing the compiler foundations.

INTENDED ARCHITECTURE



What we researched

We studied Spatial's language and implementation, mapped EE 109 patterns such as tiled memory transfers, reductions and convolution, and compared source capture, programmatic builders and compiler representations. The research is organized into language contracts, engineering blueprints, a coverage ledger and an Excalidraw architecture map on the documentation site.

The proposed design

Python syntax; explicit Spatial meaning. Ordinary Python handles host data and experiments. Kernels use a defined Python subset that the compiler reads as source. Spatial's numeric, memory and control rules are checked explicitly; captured code is not executed by Python to discover its meaning.

One program model across the system. Source, generated programs and imported artifacts will pass through the same verifier. Immutable Python records hold program meaning; each simulation keeps its own changing state. This gives checking, simulation and later hardware planning a shared reference.

Compiler-owned records as the starting point. We compared custom records with xDSL using matched structured programs and edits. The bounded results supported simpler ownership and lower representation costs for this design. xDSL remains an optional adapter if a useful backend pipeline justifies it.

Why Python is feasible

There is no theoretical need for a Rust core: the required representations, analyses and transformations can be written in Python. Correctness comes from explicit language rules, verification and independent tests. Compile time, simulation speed and memory use still need measurement on the complete workflow; the representation experiments do not settle those questions.

What is built, and what comes next

Verified checkpoint: spatial-py 092637e | October 8, 2026

Implemented compiler components

Program structure	Immutable records, source provenance, type/layout rules, scalar-expression normalization and structural checks.
Numeric rules	Signed 32-bit data arithmetic, exact integer indices and Boolean rules, with independent value and fault cases.
Memory analysis	Tracks shared backing storage, views, helper borrowing and lifetimes; records the safety and initialization obligations still to resolve.
Branches and loops	Derives conservative scalar and control facts and required arithmetic checks, including nonzero divisors and loop steps. Runtime guards are not yet inserted.

At checkpoint 092637e, all 376 component tests passed from both the source tree and an isolated installed package. Independent Codex and Opus reviews identified issues in fault attribution and resource-limit diagnostics; fixes were reproduced and regression-tested. These results validate components, not a complete compiler.

Current boundary. The complete Lab 1 source-to-simulation path has not run yet. Full verification, source capture, input preparation and reference execution remain unfinished. No Python-generated HLS or hardware result is claimed.

Next steps

- 1 Finish the verifier. Resolve safety conditions into proofs, supported runtime checks or errors; complete initialization and effect checks.
- 2 Connect source to execution. Add Python source capture, input preparation and the reference simulator. Demonstrate the Lab 1 tiled-scale case and a composed helper/branch/tail variant against independent expected results.
- 3 Test generality, then expand. Freeze the compiler and have an independent author supply a structurally unfamiliar program. Exercise a real structural edit, measure the full workflow, then extend language families and develop the HLS path from a validated subset.

What will count as success

The first milestone is a reusable family of memory programs. Changed shapes, nested loops, branches, helpers and aliases must use the same rules; no special path may recognize a lab or kernel. A successful example is evidence toward that milestone, not the entire acceptance test.

Research and evidence

[Documentation site](#) | [Current implementation status](#) | [Architecture map](#) | [Research repository](#)